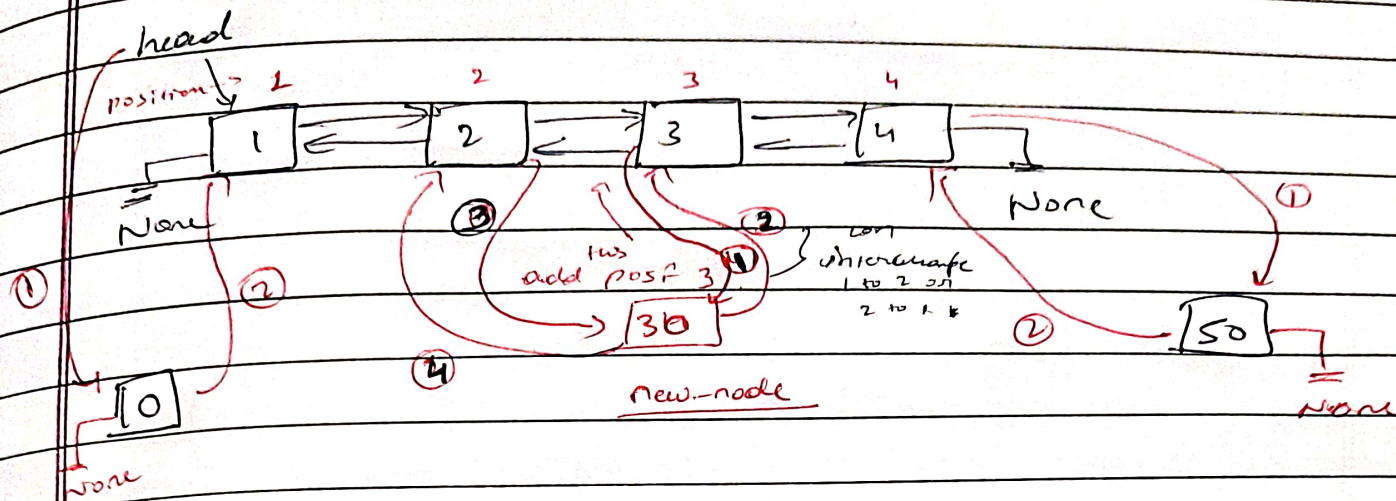


Q Doubly linked list - Insert at a Specific position

→ list empty

→ pos < 0

→ pos 10

x

x

error cond² handles.

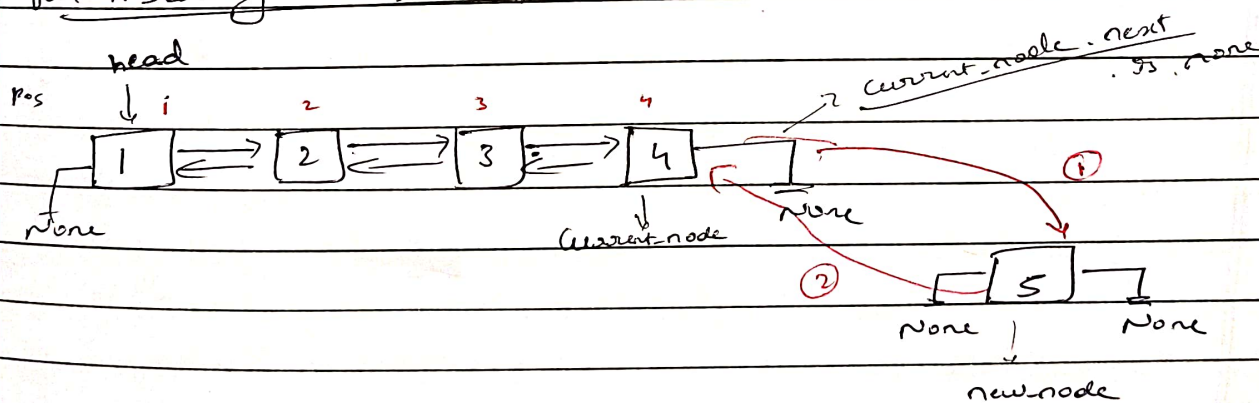
#① $\text{current-node.next}.\text{prev} = \text{new-node}$

#② $\text{new-node.next} = \text{current-node.next}$

#③ $\text{current-node.next} = \text{new-node}$

#④ $\text{new-node.prev} = \text{current-node}$

For inserting the last position



#① $\text{current-node.next} = \text{new-node}$

#② $\text{new-node.prev} = \text{current-node}$

```
7 class DLL:
10
11 def insert_at_position(self, data, position):
12
13     new_node = Node(data)
14
15     # 1. Position ≤ 0
16
17     if position <= 0:
18         print(f"{position} is invalid Position ")
19         return
20
21     # 2. Insert at beginning (position = 1)
22
23     if position == 1:
24         new_node.next = self.head      # new node points to old head      new_node.next -> head
25         if self.head:                  # if list is not empty
26             self.head.prev = new_node # old head's prev points to new node  head.prev -> new_node
27         self.head = new_node           # update head to new node          head -> new_node
28         return
29
30     # 3. Traverse to node before target
31
32     current_node = self.head
33     target_position = 1
34
35     while current_node is not None and target_position < position -1:
36         current_node = current_node.next
37         target_position += 1
38
39     # 4. Invalid (position beyond length)
40
41     if current_node is None:
42         print("invalid position beyond length ")
43         return
44
45     # Insert at the end if current.next is None
```

```

33     target_position = 1
34
35     while current_node is not None and target_position < position - 1:
36         current_node = current_node.next
37         target_position += 1
38
39     # 4. Invalid (position beyond length)
40
41     if current_node is None:
42         print(["invalid position beyond length "])
43         return
44
45     # Insert at the end if current.next is None
46
47     if current_node.next is None:
48         current_node.next = new_node
49         new_node.prev = current_node
50         return
51
52     # Insert in the middle
53
54     new_node.next = current_node.next
55     new_node.prev = current_node
56     current_node.next.prev = new_node
57     current_node.next = new_node
58

```